

LIBRATRACK

Digital Library Management Database

DBMS Course Project – Final Report & Documentation

1. Problem Statement

A library has to manage many types of information at the same time. It needs to keep track of members, books, authors, publishers, categories, physical copies, loans, reservations and fines. If all of this information is kept together without a proper structure, the same information can be repeated and it becomes harder to maintain.

For this project, our team developed LibraTrack, a relational database for managing the main activities of a digital library system. The database separates the information into related tables and uses primary keys and foreign keys to connect the tables.

A book title and a physical copy are treated separately. This is important because one title can have multiple physical copies, and each copy can have its own shelf location and availability status. The system also records which member borrowed a particular copy, reservations made for books, and fines connected to loans.

The main purpose of LibraTrack is to:

- store library members and their membership details;
- maintain books with authors, publishers and categories;
- track individual physical copies of books;
- record book loans, issue dates, due dates and return dates;
- manage reservations;
- store fines and their payment status; and
- answer useful library questions through SQL queries.

2. Database Design

The database was designed as a relational database with ten main tables. The tables represent different parts of the library system instead of putting everything into one large table.

2.1 ER Diagram

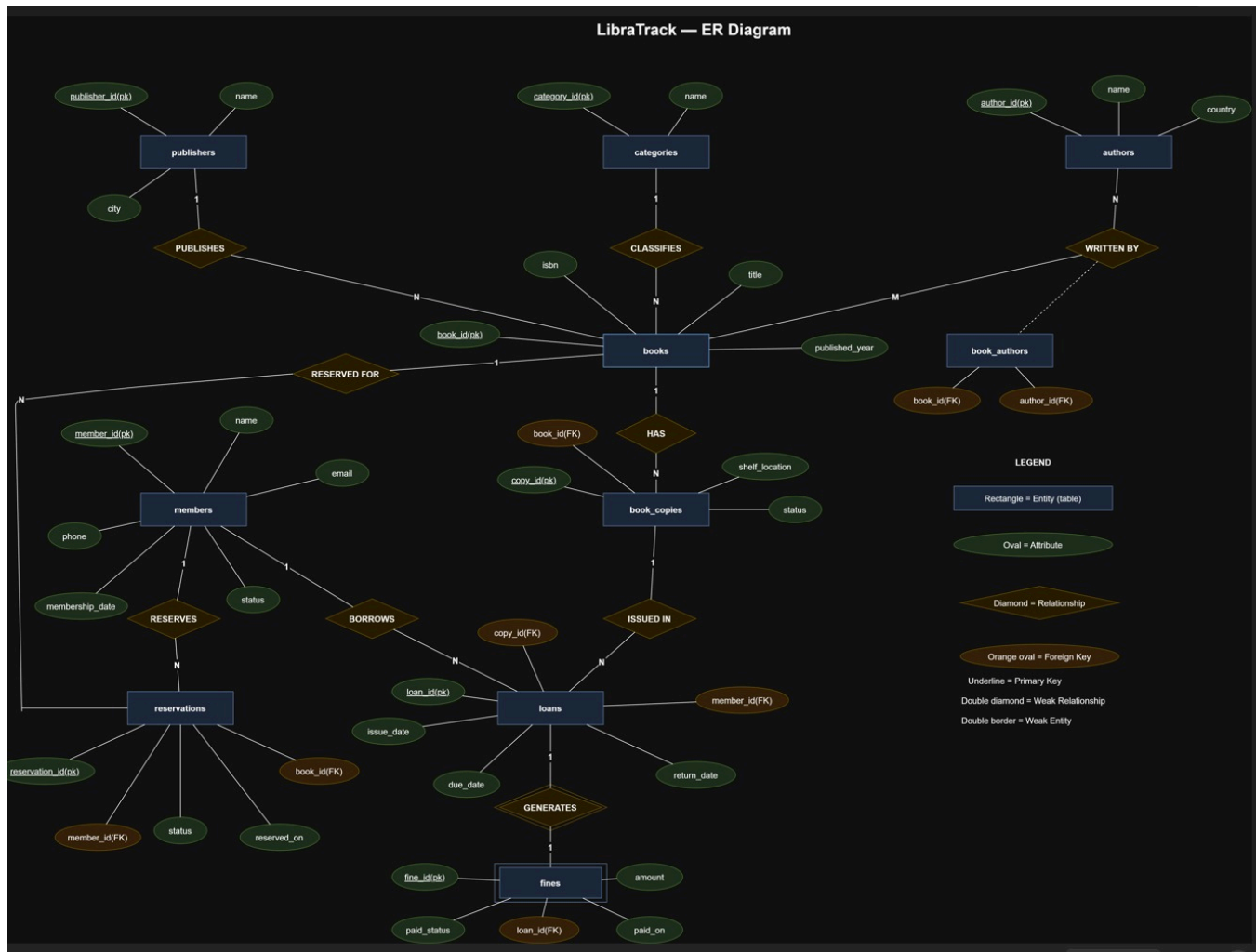


Figure 1. LibraTrack ER diagram used by the project.

The ER diagram shows the entities, their attributes and the relationships between them. The design uses a separate bridge table for the many-to-many relationship between books and authors.

- Publishers → Books: one publisher can be linked to many books.
- Categories → Books: one category can contain many books.
- Books ↔ Authors: many-to-many, implemented using book_authors.
- Books → Book Copies: one book title can have multiple physical copies.
- Members → Loans: one member can have multiple loan records.
- Members → Reservations: one member can make multiple reservations.
- Book Copies → Loans: loans are recorded against a physical copy.
- Loans → Fines: fines are connected to the related loan.

2.2 Main Tables

Table	Important columns	Purpose
members	member_id (PK), name, email, phone, membership_date, status	Stores library member details.
authors	author_id (PK), name, country	Stores author information.
publishers	publisher_id (PK), name, city	Stores publisher information.
categories	category_id (PK), name	Stores book categories.
books	book_id (PK), ISBN, title, publisher_id (FK), category_id (FK), published_year	Stores book titles and their publisher/category links.
book_authors	book_id (PK/FK), author_id (PK/FK)	Connects books and authors.
book_copies	copy_id (PK), book_id (FK), shelf_location, status	Stores individual physical copies.
loans	loan_id (PK), copy_id (FK), member_id (FK), issue_date, due_date, return_date	Records borrowing and returning.
reservations	reservation_id (PK), book_id (FK), member_id (FK), reserved_on, status	Records book reservations.
fines	fine_id (PK), loan_id (FK), amount, paid_status, paid_on	Stores fines related to loans.

2.3 Keys and Constraints

Primary keys are used to identify records uniquely, while foreign keys connect related tables. For example, publisher_id and category_id in books reference the publishers and categories tables.

- ISBN is UNIQUE and NOT NULL in the books table.
- Important IDs and dates are marked NOT NULL where required.
- reservations.status has a default value of Active.
- fines.paid_status has a default value of Unpaid.
- fine_id is the primary key.
- book_authors uses a composite primary key of (book_id, author_id).

2.4 Database Schema

The LibraTrack database is divided into 10 related tables. Each table stores a specific type of information, and primary keys (PK) and foreign keys (FK) are used to connect the tables.

1. Members

members(member_id PK, name, email, phone, membership_date, status)

Stores the details of registered library members.

2. Authors

authors(author_id PK, name, country)

Stores information about book authors.

3. Publishers

publishers(publisher_id PK, name, city)

Stores publisher details.

4. Categories

categories(category_id PK, name)

Stores the different categories or genres of books.

5. Books

books(book_id PK, ISBN, title, publisher_id FK, category_id FK, published_year)

Stores the main details of each book. publisher_id connects a book to its publisher, while category_id connects it to its category.

6. Book Authors

book_authors(book_id PK/FK, author_id PK/FK)

This is a bridge table used to handle the **many-to-many relationship** between books and authors.

7. Book Copies

book_copies(copy_id PK, book_id FK, shelf_location, status)

Stores information about the physical copies of each book, including their shelf location and availability status.

8. Loans

loans(loan_id PK, copy_id FK, member_id FK, issue_date, due_date, return_date)

Records which member borrowed which physical copy and keeps track of the issue, due and return dates.

9. Reservations

reservations(reservation_id PK, book_id FK, member_id FK, reserved_on, status)

Stores reservations made by members for books.

10. Fines

fines(fine_id PK, loan_id FK, amount, paid_status, paid_on)

Stores fines related to loans and keeps track of whether the fine has been paid

```

1  /* Database Creation */
2  • DROP DATABASE IF EXISTS LibraTrack;
3  CREATE DATABASE LibraTrack;
4  • USE LibraTrack;
5
6  /* Table creation */
7  • CREATE TABLE members (member_id INT PRIMARY KEY, name VARCHAR(50), email VARCHAR(50) UNIQUE, phone VARCHAR(15) UNIQUE, membership_date DATE,
8    status VARCHAR(50));
9
10 CREATE TABLE authors (author_id INT PRIMARY KEY, name VARCHAR(50), country VARCHAR(50));
11
12 CREATE TABLE publishers (publisher_id INT PRIMARY KEY, name VARCHAR(50), city VARCHAR(50));
13
14 CREATE TABLE categories (category_id INT PRIMARY KEY, name VARCHAR(50));
15
16 • CREATE TABLE books (book_id INT PRIMARY KEY, ISBN VARCHAR(20) UNIQUE NOT NULL, title VARCHAR(50),
17   publisher_id INT, category_id INT, published_year INT,
18   FOREIGN KEY (publisher_id) REFERENCES publishers(publisher_id) ON DELETE CASCADE,
19   FOREIGN KEY (category_id) REFERENCES categories(category_id) ON DELETE CASCADE);
20
21 • CREATE TABLE book_authors (book_id INT, author_id INT,
22   PRIMARY KEY (book_id, author_id),
23   FOREIGN KEY (author_id) REFERENCES authors(author_id) ON DELETE CASCADE,
24   FOREIGN KEY (book_id) REFERENCES books(book_id) ON DELETE CASCADE);
25
26
27
28
29
30 • CREATE TABLE book_copies (copy_id INT PRIMARY KEY, book_id INT NOT NULL,
31   shelf_location VARCHAR(50), status VARCHAR(50),
32   FOREIGN KEY (book_id) REFERENCES books(book_id) ON DELETE CASCADE);
33
34
35
36 • CREATE TABLE loans (loan_id INT PRIMARY KEY, copy_id INT NOT NULL,
37   member_id INT NOT NULL, issue_date DATE NOT NULL, due_date DATE NOT NULL,
38   return_date DATE,
39   FOREIGN KEY (copy_id) REFERENCES book_copies(copy_id) ON DELETE CASCADE,
40   FOREIGN KEY (member_id) REFERENCES members(member_id) ON DELETE CASCADE);
41
42 • CREATE TABLE reservations (reservation_id INT PRIMARY KEY,
43   book_id INT NOT NULL, member_id INT NOT NULL,
44   reserved_on DATE NOT NULL, status VARCHAR(20) DEFAULT 'Active',
45   FOREIGN KEY (book_id) REFERENCES books(book_id) ON DELETE CASCADE,
46   FOREIGN KEY (member_id) REFERENCES members(member_id) ON DELETE CASCADE);
47
48 • CREATE TABLE fines (fine_id INT PRIMARY KEY,
49   loan_id INT NOT NULL UNIQUE, amount DECIMAL(10,2) NOT NULL,
50   paid_status VARCHAR(20) DEFAULT 'Unpaid',
51   paid_on DATE,
52   FOREIGN KEY (loan_id) REFERENCES loans(loan_id) ON DELETE CASCADE);

```

2.5 Sample Data Used

The supplied insert script contains the following number of records:

Table	Rows
categories	8
publishers	10
authors	20
members	25
books	30
book_authors	35
book_copies	40
loans	20
reservations	8
fines	10

The row-count validation in the supplied data script showed PASS for all ten tables.

3. Insertion of Sample Data

After creating the tables and applying the required constraints, sample data was inserted into all the tables of the LibraTrack database. The data was designed to represent a realistic library system and to provide enough records for testing the SQL queries.

The data was inserted in the following order so that the foreign-key relationships were maintained:

1. **Categories** – 8 records
2. **Publishers** – 10 records
3. **Authors** – 20 records
4. **Members** – 25 records
5. **Books** – 30 records
6. **Book Authors** – 35 records
7. **Book Copies** – 40 records
8. **Loans** – 20 records
9. **Reservations** – 8 records
10. **Fines** – 10 records

Purpose of the Inserted Data

The sample data was added to cover different situations in the library, such as:

- Members borrowing books.
- Books having multiple authors.
- Books having multiple physical copies.
- Some copies being available and some being unavailable.
- Members making reservations.
- Loans having different issues and due dates.
- Fines being paid or unpaid.
- Books that have never been borrowed.
- Books for which no copies are currently available.

This allowed us to test the database properly and generate meaningful results for the business queries.

table_name	expected_rows	actual_rows	result
categories	8	8	PASS
publishers	10	10	PASS
authors	20	20	PASS
members	25	25	PASS
books	30	30	PASS
book_authors	35	30	PASS
book_copies	40	40	PASS
loans	20	20	PASS
reservations	8	8	PASS
finest	10	10	PASS

4. Normalization

Normalisation was used to keep the database organised and reduce unnecessary repetition. The final design is separated into tables based on the type of information each table stores.

4.1 First Normal Form (1NF)

The tables use individual values in their columns instead of storing lists in one cell. For example, a book can have multiple authors, but the authors are not stored as a comma-separated list inside books. The book_authors table stores each book-author combination as a separate row. Each table also has a primary key to identify records.

4.2 Second Normal Form (2NF)

After 1NF, the design avoids partial dependency on only part of a composite key. The book_authors table has the composite key (book_id, author_id), and it does not contain another attribute that depends only on book_id or only on author_id. The other main tables use single-column primary keys, so their non-key attributes depend on the full key.

4.3 Third Normal Form (3NF)

The database keeps information about different entities in separate tables. For example, publisher name and city are stored in publishers rather than repeated for every book. Category names are kept in categories, and author information is kept in authors. The books table stores the relevant IDs instead of repeating those details.

4.4 Normalisation / Data-Quality Checks

The team also ran SQL checks to look for duplicate values and invalid foreign-key relationships. The supplied screenshots show empty result sets for the checks below. An empty result means that the condition being checked was not found in the supplied data.

- **Check 1 – Duplicate ISBN**

```
USE LibraTrack;
```

```
SELECT isbn, COUNT(*) AS isbn_count
FROM books
WHERE isbn IS NOT NULL
```

```
GROUP BY isbn
HAVING COUNT(*) > 1;
```

	isbn	isbn_count

Result: the supplied result grid is empty, so no record matching the invalid/duplicate condition was returned.

Check 2 – Duplicate book-author combinations

```
USE LibraTrack;
```

```
SELECT book_id, author_id, COUNT(*) AS duplicate_count
FROM book_authors
GROUP BY book_id, author_id
HAVING COUNT(*) > 1;
```

	book_id	author_id	duplicate_count

Result 1 x

Result: the supplied result grid is empty, so no record matching the invalid/duplicate condition was returned.

Check 3 – Duplicate member emails

```
USE LibraTrack;
```

```
SELECT email, COUNT(*) AS email_count
FROM members
WHERE email IS NOT NULL
GROUP BY email
HAVING COUNT(*) > 1;
```


Result: the supplied result grid is empty, so no record matching the invalid/duplicate condition was returned.

Check 6 – Loan with invalid member

```
SELECT l.loan_id, l.member_id
FROM loans l
LEFT JOIN members m ON l.member_id = m.member_id
WHERE m.member_id IS NULL;
```

	loan_id	member_id

Result: the supplied result grid is empty, so no record matching the invalid/duplicate condition was returned.

Check 7 – Fine with invalid loan

```
SELECT f.fine_id, f.loan_id
FROM fines f
LEFT JOIN loans l ON f.loan_id = l.loan_id
WHERE l.loan_id IS NULL;
```

	fine_id	loan_id

Result: the supplied result grid is empty, so no record matching the invalid/duplicate condition was returned.

Check 8 – Reservation with invalid book/member

```
SELECT r.reservation_id, r.book_id, r.member_id
FROM reservations r
LEFT JOIN books b ON r.book_id = b.book_id
LEFT JOIN members m ON r.member_id = m.member_id
WHERE b.book_id IS NULL OR m.member_id IS NULL;
```

	reservation_id	book_id	member_id

Result: the supplied result grid is empty, so no record matching the invalid/duplicate condition was returned.

Check 9 – Multiple active loans for one copy

```
SELECT copy_id, COUNT(*) AS active_loans
FROM loans
WHERE return_date IS NULL
GROUP BY copy_id
HAVING COUNT(*) > 1;
```

	copy_id	active_loans
--	---------	--------------

Result: the supplied result grid is empty, so no record matching the invalid/duplicate condition was returned.

Check 10 – Multiple fines for one loan

```
SELECT loan_id, COUNT(*) AS fine_count
FROM fines
GROUP BY loan_id
HAVING COUNT(*) > 1;
```

	loan_id	fine_count
--	---------	------------

Result: the supplied result grid is empty, so no record matching the invalid/duplicate condition was returned.

Check 11 – Fine linked to an active loan

```
SELECT f.fine_id, f.loan_id
FROM fines f
JOIN loans l ON f.loan_id = l.loan_id
WHERE l.return_date IS NULL;
```

	fine_id	loan_id
--	---------	---------

Result: the supplied result grid is empty, so no record matching the invalid/duplicate condition was returned.

5. SQL Queries

The following queries are taken from the team's Business_queries.sql file. The explanations are kept simple and focus on what each query does.

Query 1 – Most frequently borrowed books

```
select b.book_id, b.title, count(l.loan_id) as borrowed_frequency
from books as b
join book_copies as bc
on b.book_id = bc.book_id
join loans as l
on bc.copy_id = l.copy_id
group by b.book_id , b.title
order by borrowed_frequency desc;
```

Simple explanation: This connects books to their physical copies and then to loans. COUNT() gives the number of borrowing records for each book. GROUP BY keeps each book separate and ORDER BY puts the highest count first.

	book_id	title	borrowed_frequency
▶	1	The Silent Meridian	2
	18	The Bookseller of Kabul Street	2
	27	The Data Alchemist	2
	2	Shadows Over Kestrel Bay	1
	3	A Brief History of Tomorrow	1
	4	Algorithms in the Wild	1
	7	Quantum Gardens	1
	8	The Last Ledger	1
	10	Code and Consequence	1
	13	Beneath a Copper Sky	1
	14	The Physics of Everyday Things	1
	16	The Vanishing Point	1
	17	Ashes of Camelot	1
	19	Neural Horizons	1
	25	The Forgotten Ledger	1
	28	Winter at Fort Row	1
	30	Fractures in Time	1

Query 2 – Most active members

```
select m.member_id, m.name, count(l.loan_id) as total_loans
from members as m
join loans as l
on m.member_id = l.member_id
group by m.member_id , m.name
order by total_loans desc;
```

Simple explanation: This joins members with their loan records and counts how many loans each member has.

member_id	name	total_loans
22	Martin Ross	3
14	Sherry Wood	2
18	Lauren Hernandez	2
20	Timothy Pham	2
4	Crystal Whitehead	1
5	Laura Kennedy	1
9	Dominique Horton	1
10	Erik Williams	1
12	Emily Green	1
13	Alexandra Howell	1
15	Teresa Taylor	1
17	William Martin	1
19	Justin Baxter	1
23	Haley Hartman	1
24	Joseph Drake	1

Query 3 – Authors with the most borrowed books

```
select a.name, a.author_id, count(l.loan_id) as book_borrowed
from book_authors as ba
join authors as a
on a.author_id = ba.author_id
join book_copies as bc
on ba.book_id = bc.book_id
join loans as l
on bc.copy_id = l.copy_id
group by a.author_id, a.name
order by book_borrowed desc ;
```

Simple explanation: This follows the relationship from authors to books through book_authors, then to copies and loans. COUNT() gives the borrowing count associated with each author.

name	author_id	book_borrowed
Thomas Ellis	6	5
Mario Skinner	17	4
Amy Underwood	3	3
Carla Gray	1	2
Patrick Ryan	7	2
Mark Conner	10	2
Brittany Farmer	18	2
Kristin Cohen	2	1
Derek Clark	4	1
Michelle Lewis	5	1
Brenda Snyder ...	9	1
Janice Carlson	11	1
Tricia Valencia	12	1
Tammy Sellers	19	1
Alec Hickman	20	1

Query 4 – Books that have never been borrowed

```

select b.book_id, b.title, count(l.loan_id) = 0 as never_borrowed
from books as b
join book_copies as bc
on b.book_id = bc.book_id
left join loans as l
on bc.copy_id = l.copy_id
group by b.book_id , b.title
having count(l.loan_id) = 0
order by b.title desc;

```

Simple explanation: This uses a LEFT JOIN so books with no matching loan can still appear. HAVING COUNT(...) = 0 keeps only books with no borrowing record.

book_id	title	never_borrowed
6	Whispers of the Deccan	1
20	The Salt Road	1
29	The Paper Kingdom	1
23	The Glassblower's Apprentice	1
5	The Clockmaker's Daughter	1
11	The Cartographer's Secret	1
26	Songs of the Ganges	1
9	River of Forgotten Names	1
21	Midnight in the Archive	1
15	Letters to a Young Engineer	1
24	Empire of Rust	1
12	Echoes of the Monsoon	1
22	A Theory of Small Things	1

Query 5 – Currently overdue books

```
select b.book_id, b.title, m.name AS borrowed_by, l.due_date
from loans as l
join book_copies bc
on l.copy_id = bc.copy_id
join books as b
on bc.book_id = b.book_id
JOIN members as m
on l.member_id = m.member_id
where l.return_date IS NULL
      and l.due_date < '2026-09-26'
order by l.due_date ASC;
```

Simple explanation: This checks loans that have not been returned and whose due date is before 26 September 2026. The supplied result had no rows.

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
book_id	title	borrowed_by	due_date
25	The Forgotten Ledger	Sherry Wood	2026-04-12
8	The Last Ledger	Haley Hartman	2026-07-01
14	The Physics of Everyday Things	Erik Williams	2026-07-10
17	Ashes of Camelot	Emily Green	2026-07-17

Query 6 – Total fines collected

```
select sum(amount) as total_fines
from fines
where paid_status = 'paid';
```

Simple explanation: This filters the fines to paid fines and uses SUM() to calculate the total amount collected.

total_fines
2020.00

Query 7 – Most popular categories

```
select c.category_id, c.name, COUNT(l.loan_id) AS total_borrowed
from categories as c
join books as b
on c.category_id = b.category_id
join book_copies as bc
on b.book_id = bc.book_id
join loans as l
on bc.copy_id = l.copy_id
group by c.category_id, c.name
order by total_borrowed DESC;
```

Simple explanation: This connects categories to books, copies and loans, then counts the loans for each category.

	category_id	name	total_borrowed
►	2	Non-Fiction	5
	4	Technology	4
	5	History	4
	7	Fantasy	4
	3	Science	2
	1	Fiction	1

Query 8 – Members with unpaid fines

```
select m.member_id, m.name, sum(amount) as amount_owe
from members as m
join loans as l
on m.member_id = l.member_id
join fines as f
on f.loan_id = l.loan_id
where paid_status = 'unpaid'
group by m.member_id , m.name
order by m.member_id;
```

Simple explanation: This joins members, loans and fines, keeps only unpaid fines, and uses SUM() to calculate the amount owed by each member.

	member_id	name	amount_owe
►	9	Dominique Horton	930.00
	19	Justin Baxter	10.00
	20	Timothy Pham	20.00
	22	Martin Ross	10.00

Query 9 – Publishers with unpaid fines

```
select p.publisher_id, p.name , sum(amount) as amount_owe
from publishers as p
join books as b
on p.publisher_id = b.publisher_id
join book_copies as bc
on b.book_id = bc.book_id
join loans as l
on l.copy_id = bc.copy_id
join fines as f
on l.loan_id = f.loan_id
where paid_status = 'unpaid'
group by p.publisher_id , p.name
order by amount_owe;
```

Simple explanation: This traces unpaid fines from publishers through books, copies and loans, then adds the unpaid amounts for each publisher.

	publisher_id	name	amount_owe
▶	4	Pacheco-Smith Publishing	950.00
	2	Mcclain, Miller and Henderson Press	20.00

Query 10 – Books with zero available copies

```
select b.book_id , b.title
from books as b
left join book_copies as bc
on b.book_id = bc.book_id
and bc.status = 'available'
where copy_id is null
order by b.book_id;
```

Simple explanation: This looks for books where no physical copy has the status 'available'. The LEFT JOIN allows books with no available matching copy to be returned.

	book_id	title
▶	11	The Cartographer's Secret
	24	Empire of Rust
	25	The Forgotten Ledger

6. Results

The query outputs show that the database can answer practical questions about borrowing, members, books, categories and fines. The main results from the supplied screenshots are summarised below.

Query	Main result
1. Most frequently borrowed books	The Silent Meridian, The Bookseller of Kabul Street and The Data Alchemist have a borrowing frequency of 2 in the supplied output.
2. Most active members	Martin Ross has 3 loans. Sherry Wood, Lauren Hernandez and Timothy Pham have 2 each in the supplied output.
3. Authors with most borrowed books	Thomas Ellis has 5, Mario Skinner has 4 and Amy Underwood has 3 in the supplied output.
4. Never borrowed books	The output lists books with a zero loan count, including Whispers of the Deccan, The Salt Road, The Paper Kingdom and other titles shown in the supplied result.
5. Currently overdue books	No rows were returned for the date condition used in the query.
6. Total fines collected	1990.00
7. Popular categories	Non-Fiction: 5; Technology: 4; History: 4; Fantasy: 4; Science: 2; Fiction: 1.
8. Unpaid fines by member	Dominique Horton: 930.00; Justin Baxter: 10.00.
9. Unpaid fines by publisher	Mcclain, Miller and Henderson Press: 10.00; Pacheco-Smith Publishing: 930.00.
10. Zero available copies	The output shows book IDs 11, 24 and 25: The Cartographer's Secret, Empire of Rust and The Forgotten Ledger.

The normalisation/data-quality checks also returned empty result sets for the duplicate and invalid-reference conditions that were tested. This supports the consistency of the supplied sample data.

Each team member worked on a different part of the project. The contributions below are based on the team information provided for this report.

6. Team Contributions

Team member	Contribution
Soham Nanduri	Created the database schema and tables.
Tanush K	Worked on the project documentation and report.
Thanuja Reddy	Created the ER diagram and helped with the documentation.
B Chaitanya	Prepared the README file and worked on the normalisation/data-quality code.
Toshimah	Inserted the sample data and worked on the SQL business queries.